

CIFAR-100 Image Classification

CNN From Scratch vs. Pretrained ResNet50 — Project Report

1. Dataset & Preprocessing

The CIFAR-100 dataset was loaded directly from `tensorflow.keras.datasets.cifar100`. It consists of 60,000 32×32 RGB images spread across 100 mutually exclusive classes, split by Keras into 50,000 training images and 10,000 test images.

The 50,000 training images were further split into a training and a validation set, reserving the first 10% (5,000 images) for validation and keeping the remaining 45,000 images for training. No separate normalization step was applied before this split; instead, pixel rescaling (1/255) was built directly into the scratch CNN as a **Rescaling** layer, while the ResNet50 branch used the ImageNet-specific `preprocess_input` function together with a resize from 32×32 to 94×94, matching the input resolution ResNet50 expects.

Split	Images	Shape
Training	45,000	(45000, 32, 32, 3)
Validation	5,000	(5000, 32, 32, 3)
Test	10,000	(10000, 32, 32, 3)

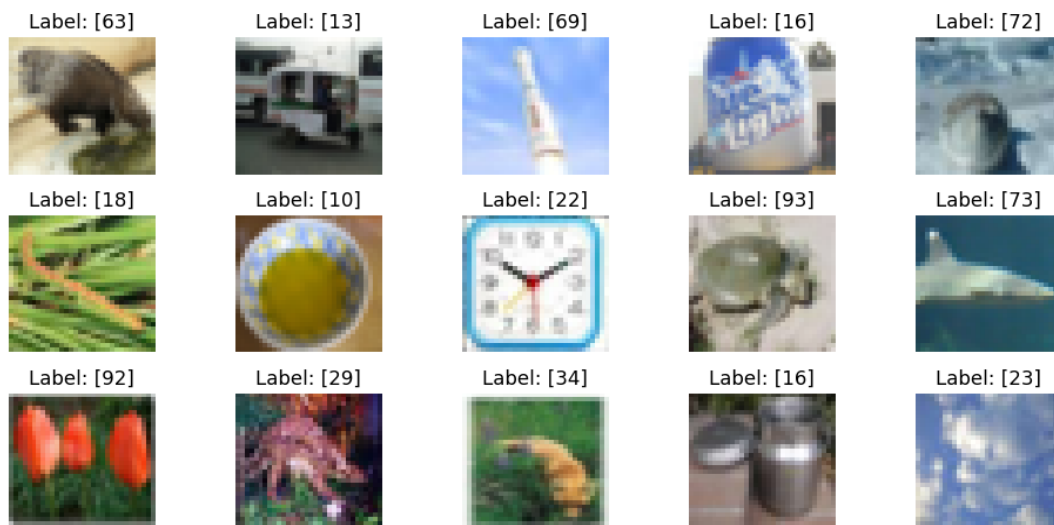


Figure 1. Random sample of 15 labeled CIFAR-100 training images.

2. CNN Model From Scratch

A deep sequential CNN (**Scratch_CNN_Model**) was designed and trained from randomly initialized weights. The architecture stacks eight convolutional blocks of decreasing/varying width, each followed by Batch

Normalization and Dropout for regularization, before flattening into a three-layer dense classification head.

Architecture summary

- Input: 32×32×3 images, rescaled to [0, 1] via a **Rescaling(1/255)** layer
- Conv Block 1: Conv2D(32, 4×4, ReLU) → BatchNorm → MaxPool2D → Dropout(0.1)
- Conv Block 2: Conv2D(128, 4×4, ReLU) → BatchNorm → MaxPool2D → Dropout(0.3)
- Conv Block 3: Conv2D(256, 4×4, ReLU) → BatchNorm → MaxPool2D → Dropout(0.3)
- Conv Block 4: Conv2D(80, 4×4, ReLU) → BatchNorm → MaxPool2D → Dropout(0.15)
- Conv Block 5: Conv2D(64, 4×4, ReLU) → BatchNorm → Dropout(0.16)
- Conv Block 6: Conv2D(60, 3×3, ReLU) → BatchNorm → Dropout(0.1)
- Conv Block 7: Conv2D(55, 3×3, ReLU) → BatchNorm → Dropout(0.1)
- Conv Block 8: Conv2D(32, 3×3, ReLU) → BatchNorm → Dropout(0.05)
- Flatten → Dense(128, ReLU) → BatchNorm → Dropout(0.2)
- Dense(64, ReLU) → BatchNorm → Dropout(0.2)
- Dense(16, ReLU) → BatchNorm → Dropout(0.1)
- Output: Dense(100, Softmax)

Training configuration

- Optimizer: Adam (default learning rate)
- Loss: Sparse Categorical Crossentropy
- Metric: Accuracy
- Epochs: up to 50, with EarlyStopping (monitor = val_loss, patience = 10, restore_best_weights = True)
- Total parameters: 1,112,935 (1,111,105 trainable / 1,830 non-trainable)

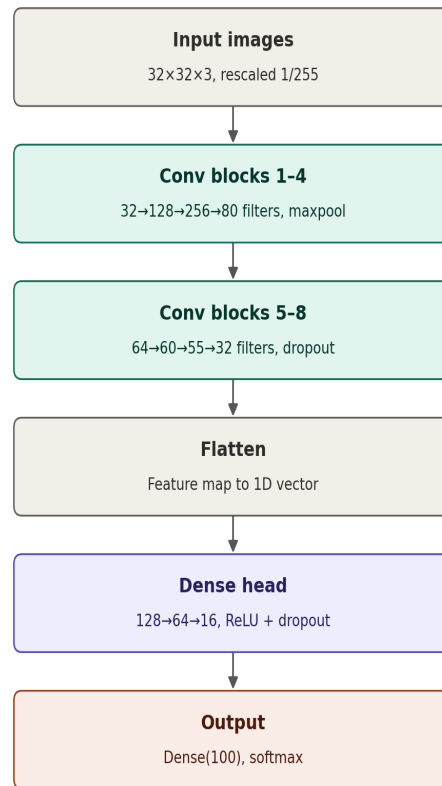


Figure A. Scratch CNN — data flow through the network.

3. Pretrained Model — ResNet50

For transfer learning, **ResNet50** was loaded from **tensorflow.keras.applications** with ImageNet weights, **include_top=False** and **pooling='avg'**, producing a 2048-dimensional feature vector per image. The convolutional base was fully frozen (**base_model.trainable = False**), so only a new classification head was trained on top of the fixed ImageNet features (feature extraction rather than fine-tuning).

Because ResNet50 expects larger inputs than CIFAR-100's native 32x32 resolution, images were resized to 94x94 and passed through the model-specific **preprocess_input** function using a **tf.data** pipeline (resize → preprocess → cache → shuffle → batch(300) → prefetch) for efficient training.

Classification head

- ResNet50 base (frozen, 23,587,712 non-trainable params) → Dense(128, ReLU) → Dropout(0.3)
- Dense(64, ReLU) → Dropout(0.2)
- Dense(32, ReLU) → Dropout(0.15)
- Dense(16, ReLU) → Dropout(0.1)
- Output: Dense(100, Softmax)

Training configuration

- Optimizer: Adam · Loss: Sparse Categorical Crossentropy · Metric: Accuracy
- Epochs: up to 50, with EarlyStopping (monitor = val_loss, patience = 10, restore_best_weights = True)
- Total parameters: 23,862,548 (274,836 trainable / 23,587,712 non-trainable, i.e. only ~1.15% of parameters were updated)

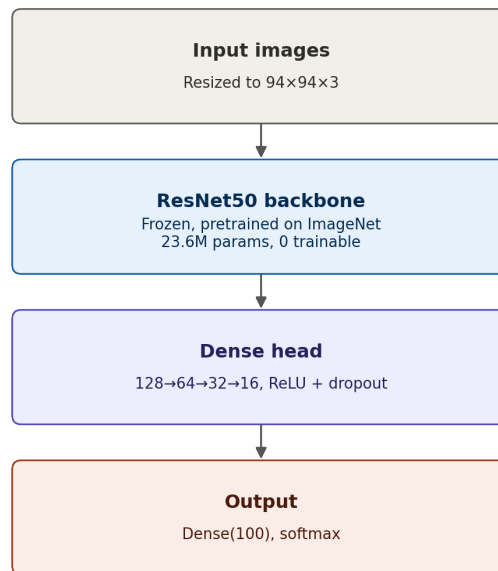


Figure B. ResNet50 transfer learning — data flow through the network.

4. Evaluation & Results

Metric	Scratch CNN	ResNet50
Training Accuracy	50.74%	56.34%
Validation Accuracy	42.22%	57.76%
Test Accuracy	42.96%	57.07%
Training Loss	1.7659	1.4387
Validation Loss	2.3207	1.6965
Test Loss	2.2278	1.6633

Both models were trained with early stopping on validation loss (patience = 10 epochs), so the reported training/validation figures correspond to the restored best-weight epoch, and test figures come from a final evaluation on the untouched 10,000-image test set.

Training curves — Scratch CNN

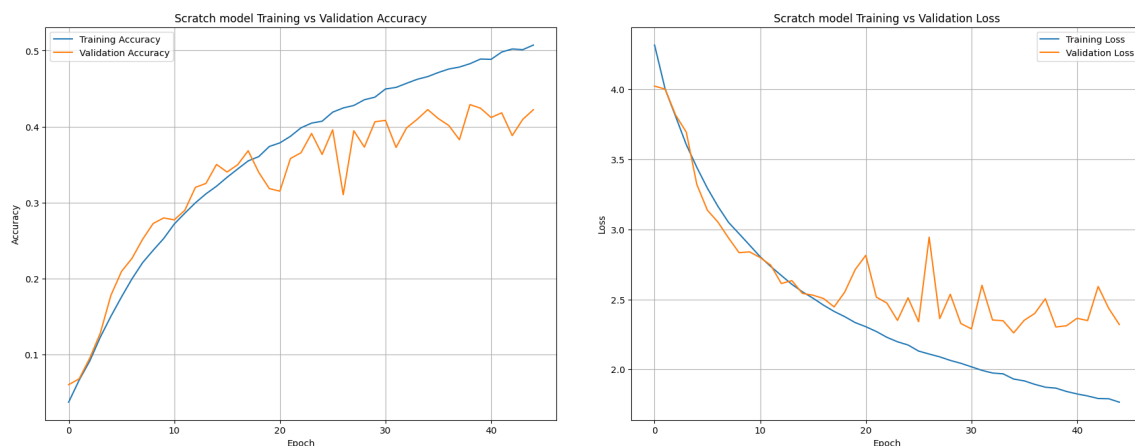


Figure 2. Scratch CNN — Training vs. Validation Accuracy (left) and Loss (right).

Training curves — ResNet50

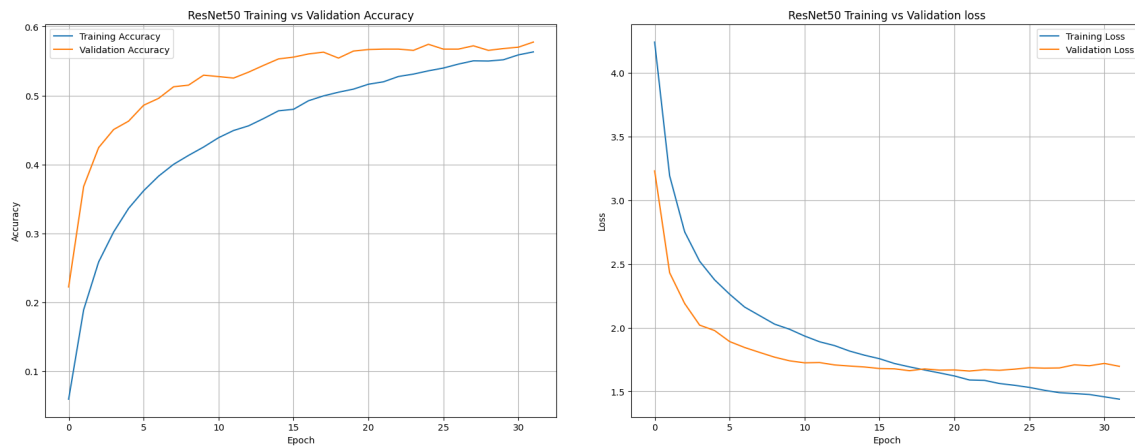


Figure 3. ResNet50 — Training vs. Validation Accuracy (left) and Loss (right).

5. Comparison

Metric	Scratch CNN	ResNet50
Test Accuracy	42.96%	57.07%
Test Loss	2.2278	1.6633
Number of Parameters	1,112,935	23,862,548
Trainable Parameters	1,111,105	274,836
Training Time	638 seconds	1000 seconds
Overall Performance	Moderate — plateaus around 43%	Strong — clear margin over scratch CNN

ResNet50 outperforms the scratch CNN on every accuracy/loss metric despite having roughly 21× more total parameters, because the vast majority of those parameters (the frozen 23.6M-parameter backbone) were pretrained on ImageNet and never had to be learned from the 45,000 CIFAR-100 training images. Only 274,836 new parameters — about 1.15% of the full model — were actually trained. Its longer wall-clock training time reflects the added cost of running the larger backbone and image resizing pipeline, not more optimization steps.

6. Conclusion

Overall, the **ResNet50 pretrained model performed better than the CNN built from scratch**. It reached a higher validation accuracy (57.76% vs. 42.22%) and test accuracy (57.07% vs. 42.96%), while also achieving a lower test loss (1.6633 vs. 2.2278). Although ResNet50 took longer to train (1000 vs. 638 seconds) and carries far more total parameters, its stronger performance highlights the value of transfer learning.

ResNet50's advantage comes from starting with convolutional features already learned on the large and diverse ImageNet dataset, rather than learning every filter from CIFAR-100's relatively small 45,000-image training set. This gives the pretrained model a strong, general-purpose visual representation from the outset, which tends to improve accuracy, generalization, and convergence speed per epoch — especially on a 100-class problem where a from-scratch model has comparatively little data per class to learn from. For this task, **ResNet50 is the better-performing model**, and using pretrained backbones is a strong default strategy whenever labeled data is limited relative to the number of classes.